

Python untuk Big Data: Polars Deep Dive

Pertemuan 2 — Analisis Big Data

S1 Teknik Informatika / Sistem Informasi | Semester Ganjil 2026

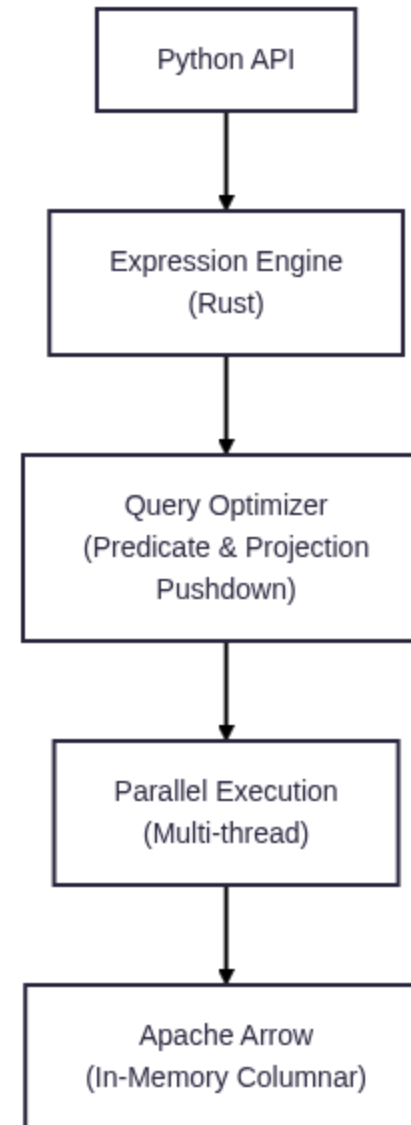
Agenda Pertemuan 2

1. Mengapa Polars? Arsitektur Rust + Apache Arrow
2. Lazy vs Eager API: cara berpikir yang berbeda
3. Polars Expressions: cara bekerja kolumnar
4. filter, select, with_columns
5. group_by dan agregasi
6. Joins di Polars
7. Window Functions
8. Lazy loading: scan_parquet / scan_csv
9. Menulis Parquet dengan partisi

Tugas 1 Milestone 1 diumumkan hari ini!

Mengapa Polars?

Arsitektur Internal Polars



Lazy vs Eager API

Eager API (seperti Pandas)

```
# Langsung dieksekusi – hasil tersedia
df = pl.read_parquet("data.parquet")
# Baca SEMUA ke RAM

hasil = (
    df
    .filter(pl.col("status") == "selesai")
    .group_by("kota")
    .agg(pl.col("harga").sum())
)
```

Masalah: data 10GB = butuh RAM 10GB+

```
# Bangun rencana – belum eksekusi
hasil = (
    pl.scan_parquet("data.parquet")
    # Predicate pushdown: filter
    # diterapkan SAAT membaca file
    .filter(pl.col("status") == "selesai")
    # Projection pushdown: hanya
    # baca 2 kolom dari file
    .select(["kota", "harga"])
    .group_by("kota")
    .agg(pl.col("harga").sum())
    .collect() # Baru eksekusi!
)
```

Lazy API (dioptimalkan)

Melihat Query Plan

```
q = (  
    pl.scan_parquet("data.parquet")  
    .filter(pl.col("status") == "selesai")  
    .group_by("kota")  
    .agg(pl.col("harga").mean())  
)  
  
# Tampilkan rencana yang dioptimalkan  
print(q.explain(optimized=True))
```

Output:

```
AGGREGATE  
  [col("harga").mean()] BY [col("kota")] FROM  
  FILTER [(col("status")) == (String(selesai))] FROM  <-- filter diturunkan  
    Parquet SCAN [data.parquet]  
    PROJECT 2/9 COLUMNS                               <-- hanya 2 dari 9 kolom
```

Polars secara otomatis mengoptimalkan urutan operasi. Tulis query yang mudah dibaca, Polars yang mengoptimalkan.

Polars Expressions

Konsep kunci: Polars bekerja pada **kolom**, bukan baris. Operasi dijalankan secara paralel.

```
# select: pilih dan transformasi kolom
df.select([
    pl.col("nama"),
    pl.col("harga"),
    # Kolom baru dari komputasi
    (pl.col("harga") * pl.col("kuantitas")).alias("total"),
    # Kondisional dengan when/then/otherwise
    pl.when(pl.col("harga") > 1_000_000)
        .then(pl.lit("Premium"))
        .otherwise(pl.lit("Reguler"))
        .alias("segmen"),
])
```

```
# with_columns: tambah kolom tanpa hapus yang lain
df.with_columns([
    (pl.col("harga") * 1.11).alias("harga_dengan_pajak"),
    pl.col("kota").str.to_uppercase().alias("kota_upper"),
    pl.col("tanggal").dt.month().alias("bulan"),
])
```

filter: Kondisi dan Logika Boolean

```
# Filter satu kondisi
df.filter(pl.col("status") == "selesai")

# Filter multi kondisi (AND, OR, NOT)
df.filter(
    (pl.col("status") == "selesai") &
    (pl.col("harga") > 500_000) &
    (pl.col("kota").is_in(["Jakarta", "Surabaya"]))
)

# Filter dengan string operations
df.filter(
    pl.col("kategori").str.to_lowercase().str.contains("elekt")
)

# Filter dengan null check
df.filter(pl.col("rating").is_not_null())

# Filter berdasarkan quantile
batas = df["harga"].quantile(0.95)
df.filter(pl.col("harga") <= batas)
```

group_by dan Agregasi

```
# Agregasi tunggal
df.group_by("kategori").agg(
    pl.col("harga").sum().alias("total_penjualan")
)

# Agregasi lengkap – banyak metrik sekaligus
df.group_by(["kategori", "kota"]).agg([
    pl.col("transaksi_id").count().alias("jumlah"),
    pl.col("harga").sum().alias("total"),
    pl.col("harga").mean().alias("rata_rata"),
    pl.col("harga").median().alias("median"),
    pl.col("harga").std().alias("standar_deviasi"),
    pl.col("rating").mean().round(2).alias("avg_rating"),
    # Kondisional dalam agregasi
    (pl.col("status") == "selesai").sum().alias("transaksi_berhasil"),
])
.sort("total", descending=True)
```

Joins di Polars

Jenis Join

Jenis	Deskripsi	Kata kunci
Inner	Baris ada di kedua tabel	<code>how="inner"</code>
Left	Semua baris kiri	<code>how="left"</code>
Right	Semua baris kanan	<code>how="right"</code>
Full Outer	Semua baris kedua tabel	<code>how="full"</code>
Anti	Baris kiri TIDAK ada di kanan	<code>how="anti"</code>
Semi	Baris kiri ADA di kanan	<code>how="semi"</code>

Contoh Kode

```
# Tabel dimensi kategori
dim_kategori = pl.DataFrame({
    "kategori": ["Elektronik", "Fashion"],
    "divisi": ["Teknologi", "Gaya Hidup"],
    "margin_persen": [12, 35],
})

# Inner join
df.join(
    dim_kategori,
    on="kategori",
    how="inner"
)

# Join pada kolom berbeda nama
df.join(
    dim_kota,
    left_on="kota_penjual",
    right_on="nama_kota",
    how="left"
```

Kuis Checkpoint 1 (Menit 25)

Buka Wayground — Instructor-Paced Mode

3 soal konseptual Polars API:

1. Apa keuntungan utama menggunakan LazyFrame dibandingkan DataFrame biasa pada dataset berukuran sangat besar?
2. Perbedaan mendasar antara `select()` dan `with_columns()` pada Polars adalah...
3. Dalam Polars, operasi `group_by().agg()` berbeda dari SQL GROUP BY karena...

Waktu: 3 menit

Window Functions

Window functions menghasilkan agregasi **tanpa mereduksi jumlah baris**.

```
df.with_columns([\n    # Ranking dalam grup (over = partition by)\n    pl.col("penjualan")\n        .rank(method="dense", descending=True)\n        .over("kota")\n        .alias("rank_dalam_kota"),\n\n    # Running total (kumulatif)\n    pl.col("penjualan")\n        .cum_sum()\n        .over("kota")\n        .alias("kumulatif_penjualan"),\n\n    # Rata-rata grup (tanpa reduksi baris)\n    pl.col("penjualan")\n        .mean()\n        .over("kategori")\n        .alias("rata_rata_kategori"),\n\n    # Persentase terhadap total grup\n    (pl.col("penjualan") /\n     pl.col("penjualan").sum().over("kota") * 100)\n        .round(1)\n        .alias("persen_kontribusi"),\n])
```

scan_parquet: Lazy Loading

```
# Bayangkan file 50GB – tidak muat di RAM!
# scan_parquet membangun rencana tanpa membaca file

hasil = (
    pl.scan_parquet("data_besar/*.parquet") # Glob: baca banyak file
    .filter(pl.col("tahun") == 2024)         # Hanya baris 2024
    .filter(pl.col("provinsi") == "DKI Jakarta")
    .select(["kategori", "harga", "status"]) # Hanya 3 dari 20 kolom
    .filter(pl.col("status") == "selesai")
    .group_by("kategori")
    .agg(pl.col("harga").sum())
    .collect() # Di sinilah file benar-benar dibaca
)
```

Yang terjadi saat `.collect()` :

1. Optimizer menggabungkan semua filter
2. Hanya 3 kolom yang dibaca dari file
3. Filter diterapkan saat membaca (predicate pushdown)
4. Hasilnya jauh lebih kecil dari file asli
5. Pemrosesan paralel di semua CPU core

Kuis Checkpoint 2 (Menit 55)

Buka Wayground — Instructor-Paced Mode

3 soal praktikal Polars Expressions dan Kode:

4. Perhatikan kode berikut: `df.filter(pl.col("harga") > 0).select(["kota", "harga"]).group_by("kota").agg(pl.col("harga").sum())` — urutan mana yang dioptimalkan oleh Polars Lazy?
5. Untuk menambahkan kolom baru ke DataFrame tanpa menghapus kolom yang sudah ada, fungsi Polars yang tepat adalah...
6. Window function `.over("kota")` dalam Polars setara dengan klausa SQL apa?

Waktu: 3 menit

Menulis Parquet dengan Partisi

Partisi memungkinkan DuckDB/Polars hanya membaca file yang relevan:

```
# Tulis dengan partisi per kategori dan bulan
df.with_columns(
    pl.col("tanggal").dt.month().alias("bulan")
).write_parquet(
    "output/transaksi/",
    use_pyarrow=True,
    pyarrow_options={"partition_cols": ["kategori", "bulan"]},
)
```

Struktur folder yang dihasilkan:

```
output/transaksi/
├── kategori=Elektronik/
│   ├── bulan=1/data.parquet
│   ├── bulan=2/data.parquet
│   └── ...
├── kategori=Fashion/
│   ├── bulan=1/data.parquet
│   └── ...
```

Query hanya `kategori=Elektronik` = hanya 1/7 folder yang dibaca!

String dan Temporal Operations

String Operations

```
df.with_columns([
    # Strip spasi
    pl.col("nama").str.strip_chars(),
    # Uppercase/lowercase
    pl.col("kota").str.to_titlecase(),
    # Contains
    pl.col("email").str.contains("@gmail"),
    # Split
    pl.col("nama_lengkap").str.split(" ")
        .list.first().alias("nama_depan"),
    # Replace
    pl.col("kode").str.replace("ID-", ""),
    # Regex
    pl.col("email").str.contains(
        r"^\[\\w.]+@[\\w.]+\\.\\w{2,}$"
    ).alias("email_valid"),
])
```

Temporal Operations

```
df.with_columns([
    pl.col("tanggal").dt.year(),
    pl.col("tanggal").dt.month(),
    pl.col("tanggal").dt.day(),
    pl.col("tanggal").dt.weekday(),
    pl.col("tanggal").dt.quarter(),
    # Selisih waktu
    (pl.col("tanggal_selesai") -
     pl.col("tanggal_mulai"))
        .dt.total_minutes()
        .alias("durasi_menit"),
    # Resampling (group_by_dynamic)
    # Dibahas di P06 Time Series
])
```

Kuis Checkpoint 3 (Menit 85)

Buka Wayground — Instructor-Paced Mode

4 soal sintesis dan optimasi:

7. Anda memiliki file Parquet 30GB berisi 500 kolom. Anda hanya butuh 3 kolom dan 10% baris. Pendekatan Polars yang paling efisien adalah...
8. Perhatikan: `df.lazy().filter(A).filter(B).select(C)` . Optimizer Polars akan melakukan apa?
9. Window function `.over("kota")` pada DataFrame 5 juta baris — kapan ini lebih efisien dari `group_by("kota").agg(...).join(df, on="kota")` ?
10. Dalam hal penggunaan memori, perbedaan utama antara Polars Arrow dan Pandas NumPy adalah...

Waktu: 4 menit

Ringkasan Pertemuan 2

Polars adalah DataFrame engine modern berbasis Rust dan Apache Arrow yang:

- Menggunakan **Lazy API** untuk optimasi query otomatis
- Bekerja **kolumnar dan paralel** untuk performa tinggi
- Mendukung **Expressions** yang ekspresif dan composable
- Memiliki **Window Functions** untuk analitik tanpa reduksi baris
- Membaca data efisien dengan **predicate dan projection pushdown**

Tugas 1 Milestone 1: Clone repo GitHub Classroom, akuisisi dataset >500MB/1M baris

Deadline: Sebelum Pertemuan 3

Pertemuan Berikutnya: DuckDB & Format Data — SQL analitik in-process

Referensi

- Polars User Guide: <https://docs.pola.rs/user-guide/>
- Polars API Reference: <https://docs.pola.rs/api/python/stable/reference/>
- Janssens, J., & Nieuwdorp, T. (2025). *Python Polars: The Definitive Guide*. O'Reilly.